1 A.1 Compiler and Linker Flags

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding -std=gnu17
CFLAGS += -Wall -Wextra -O0 -g3

LDFLAGS := -T scripts/mps2_m3.ld
LDFLAGS += -Xlinker -Map=build/output.map
LDFLAGS += -nostdlib -lgcc
```

Flag	Meaning
`-mthumb -mcpu=cortex-m3`	Selects the Thumb instruction subset implemented by the Cortex-M3
`-nostartfiles`	Excludes the toolchain's default C startup objects, so reset handling is done by the project's own startup code
`-ffreestanding`	Compiles for a freestanding (bare-metal) C environment, without hosted-library assumptions
`-std=gnu17`	C17 with GNU extensions
`-Wall -Wextra`	Warning policy
`-O0 -g3`	No optimisation and full debug information: a debuggable, source-corresponding image
`-T scripts/mps2_m3.ld`	Uses the given linker script to describe memory and section placement
`-Xlinker -Map=build/output.map`	Makes the linker emit a map file of every placement and symbol address
`-nostdlib -lgcc`	Links no standard library, only `libgcc` for compiler support routines

> **DIAGNOSING A BUILD FAILURE**
>
> The full command line is the exact contract that produced the failure: copy it before changing anything.

2 A.2 Running and Debugging the Image

2.1 A.2.1 Running the Image in QEMU

QEMU **emulates** the target board on the host, so the image can run before any physical hardware is used:

```
qemu-system-arm -machine mps2-an385 -kernel build/scratch-os.axf -nographic -semihosting
```

Option	Meaning
`-machine mps2-an385`	The board to emulate (here a Cortex-M3): its memory map must match the linker script
`-kernel`	The ELF image to boot

Option	Meaning
`-nographic`	Routes the target console to the host terminal
`-semihosting`	Lets the program print through the host (see below)

> ⚠ **QEMU DOES NOT EXIT**
>
> A bare-metal program has no operating system to return to: when `main()` returns, the startup code enters an infinite loop, so QEMU keeps running and must be stopped by hand.

2.2 A.2.2 Semihosting: Output Through the Host

A bare-metal program has **no console**: `printf` needs a C library and an operating system, and a UART needs a driver. **Semihosting** solves this during **development**: the program asks the **host**, the computer running QEMU or a debugger, to perform the output on its behalf.

The request follows a convention defined by ARM:

Element	Role	For printing a string
`r0`	Which service is requested	`0x04` (`SYS_WRITE0`)
`r1`	The argument of the service	Address of the string
`bkpt 0xab`	Signals that the request is ready	—

When the CPU executes `bkpt 0xab`, QEMU stops it, reads `r0` and `r1`, prints the string on the host terminal, and resumes the program at the next instruction. This is a **trap**: control passes to a more powerful entity, which performs the service and returns. A system call has the same shape, with the kernel in the role of the host.

```
static void semihost_write0( const char *message )
{
    register uint32_t    r0 __asm__( "r0" ) = 0x04;    /* SYS_WRITE0 */
    register const char *r1 __asm__( "r1" ) = message; /* string address */
    __asm__ volatile ( "bkpt 0xab" : : "r"( r0 ), "r"( r1 ) : "memory" );
}
```

C cannot choose registers nor emit `bkpt`, so the function uses two GCC extensions: `register … __asm__( "r0" )` forces a variable into a register, and inline assembly inserts the instruction (`volatile` stops the compiler from removing it).

> ⚠ **DEVELOPMENT TOOL ONLY**
>
> Semihosting works only when QEMU or a debugger is listening. On a real board with nothing attached, `bkpt` simply halts the CPU: production firmware prints through a peripheral such as a UART.

2.3 A.2.3 Debugging with `gdb`

A **debugger** stops the program, executes it one instruction or one line at a time, and shows registers and memory. On a microcontroller, where a crash often prints nothing, it is often the only way to see what the CPU does.

Debugging is **remote**: the program runs in QEMU, started with two extra options, and **gdb connects to it over TCP**:

- `-S`: freeze the CPU before its first instruction;
- `-gdb tcp::1234`: wait for gdb on port 1234.

gdb (`arm-none-eabi-gdb`, or `gdb-multiarch` on Linux) is started on the same ELF, from which it reads only symbols and debug information.

Command	Effect
`target remote :1234`	Connect to QEMU
`info registers sp pc`	Show registers
`x/2wx 0`	Examine 2 words in hex at address 0
`p/x $r1`	Print a register or an expression in hex
`stepi` (`si`)	Execute one machine instruction
`break main` (`b`)	Stop when execution reaches `main`
`continue` (`c`)	Run to the next breakpoint

2.4 A.2.4 Locating a Failure

Each kind of failure points to a **different stage** of the pipeline:

Symptom	Where to look
The compiler rejects a command	Toolchain installation and Makefile flags
Unresolved symbol	Object files on the link command and matching declarations
Section overflow	Memory regions in the linker script, map file
QEMU runs but prints nothing	Reset path, output call, `-semihosting` option; attach gdb to see how far execution gets

> **LOCATE THE FAILING STAGE**
>
> Read the **first** diagnostic and identify which stage failed before changing anything: changing compiler flags at random is not a diagnosis.

3 A.3 Build Automation with Make

`make` is not a compiler: it reads a `Makefile`, which describes how each file of the build is produced, and runs only the commands needed to bring the requested files up to date.

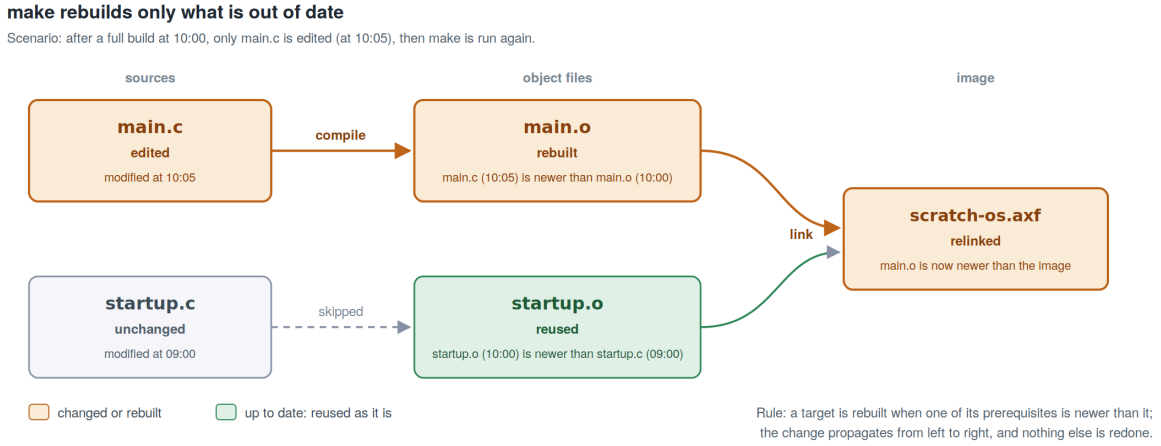
3.1 A.3.1 Rules and Rebuilds

A `Makefile` is a list of **rules**:

```
target: prerequisite1 prerequisite2
        recipe line 1
        recipe line 2
```

Part	Meaning
`target`	The file to produce, such as an object file or the ELF image
`prerequisites`	The files the target is made from; `make` brings them up to date first
`recipe`	The shell commands that produce the target; every line **starts with a Tab**, not spaces

A target is rebuilt only if it does not exist or if one of its prerequisites is **newer** than it (by modification time). Since prerequisites can be targets of other rules, the rules form a dependency graph, and a change propagates along it:



The rebuild is correct only if the graph lists every real dependency. A header or a linker script that is not listed as a prerequisite does not trigger a rebuild when it changes: `make clean && make` forces a full one.

3.2 A.3.2 The Tab Rule

A recipe line indented with spaces stops `make` with an error that names the line but not the cause:

```
$ make
Makefile:5: *** missing separator.  Stop.
```

Spaces and a Tab look the same in an editor; `cat -et` shows the difference by printing each Tab as `^I` and each line end as `$`:

```
$ cat -et Makefile
$(BUILD_DIR)/%.o: %.c$
    @mkdir -p $(dir $@)$
```

```
^I$(CC) $(CFLAGS) -c $< -o $@$
```

The first recipe line starts with four spaces, the second with a Tab (`^I`). Many editors insert spaces when the Tab key is pressed: configure them to keep real Tabs in Makefiles.

3.3 A.3.3 Variables

```
CC      := arm-none-eabi-gcc
CFLAGS  := -mthumb -mcpu=cortex-m3
CFLAGS  += -Wall -Wextra
```

Operator	Behaviour
`:=`	Evaluates the right-hand side once, immediately: the usual choice
`=`	Re-evaluates the right-hand side at every use
`+=`	Appends to the current value
`?=`	Assigns only if the variable is not already set, so it can be overridden from the command line (`make CC=clang`)

A variable is used with `$(NAME)` . By convention, `CC` , `CFLAGS` , `LDFLAGS` , and `LDLIBS` hold the compiler and its options; other names, such as `SOURCES` or `BUILD_DIR` , are the Makefile's own.

3.4 A.3.4 Pattern Rules and Automatic Variables

One **pattern rule** compiles every source file, instead of one rule per file:

```
$(BUILD_DIR)/%.o: %.c
	@mkdir -p $(dir $@)
	$(CC) $(CFLAGS) -c $< -o $@
```

`%` matches the same text on both sides: for `build/src/main.o` it matches `src/main` , so the prerequisite is `src/main.c` . Inside the recipe, **automatic variables** name the files of the rule being run:

Variable	Meaning
`$@`	The target
`$<`	The first prerequisite
`$^`	All the prerequisites
`$(dir $@)`	The directory of the target, created with `mkdir -p` before the compiler writes into it

The list of objects is derived from the list of sources with a **substitution**: `$(SOURCES:%.c=$(BUILD_DIR)/%.o)` turns `src/main.c` into `build/src/main.o` . The source list can be written by hand or collected with `$(wildcard src/*.c)` : the explicit list is easier to check, the wildcard picks up new files automatically.

3.5 A.3.5 Phony Targets

```
.PHONY: all run clean
```

`run` and `clean` are actions, not files. Without `.PHONY`, a file called `clean` in the directory would make `make` think the target is up to date and skip it. `all`, the first target of the file, is the **default goal**, built when `make` is run with no arguments.

3.6 A.3.6 Debugging a Makefile

A Makefile can be wrong without failing, for example when a substitution builds the wrong list. What matters is what the variables expand to:

Tool	Effect
`$(info OBJECTS = $(OBJECTS))`	Prints a value while the Makefile is read
`$(warning …)`	Same, with the `file:line:` of the call
`make -n`	Prints the commands it would run, without running them

3.7 A.3.7 A Complete Makefile

```
CC       := arm-none-eabi-gcc
CFLAGS   := -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding -std=gnu17
CFLAGS   += -Wall -Wextra -O0 -g3
LDFLAGS  := -T scripts/mps2_m3.ld
LDFLAGS  += -Xlinker -Map=build/output.map
LDFLAGS  += -nostdlib -lgcc

BUILD_DIR := build
BIN       := $(BUILD_DIR)/scratch-os.axf
SOURCES   := boot/startup.c src/main.c
OBJECTS   := $(SOURCES:%.c=$(BUILD_DIR)/%.o)

.PHONY: all run clean
all: $(BIN)

$(BUILD_DIR)/%.o: %.c
        @mkdir -p $(dir $@)
        $(CC) $(CFLAGS) -c $< -o $@

$(BIN): $(OBJECTS)
        $(CC) $(CFLAGS) $(LDFLAGS) $(OBJECTS) -o $(BIN)
```

```
run: $(BIN)
	qemu-system-arm -machine mps2-an385 -kernel $(BIN) -nographic -semihosting

clean:
	rm -rf $(BUILD_DIR)
```

`make` compiles the stale objects and links them; run again with no changes, it does nothing. `make run` also boots the image in QEMU, and `make clean` deletes the `build/` directory, which holds every generated file.

> **LIBRARY ORDER**
>
> The linker takes from a library only the functions requested by the objects read **before** it. With `-lgcc` in `LDFLAGS`, which comes before the objects on the command line, code that needs a libgcc helper (such as a 64-bit division, `__aeabi_uldivmod`) fails to link. Moving `-lgcc` after the objects fixes it.